

Introdução

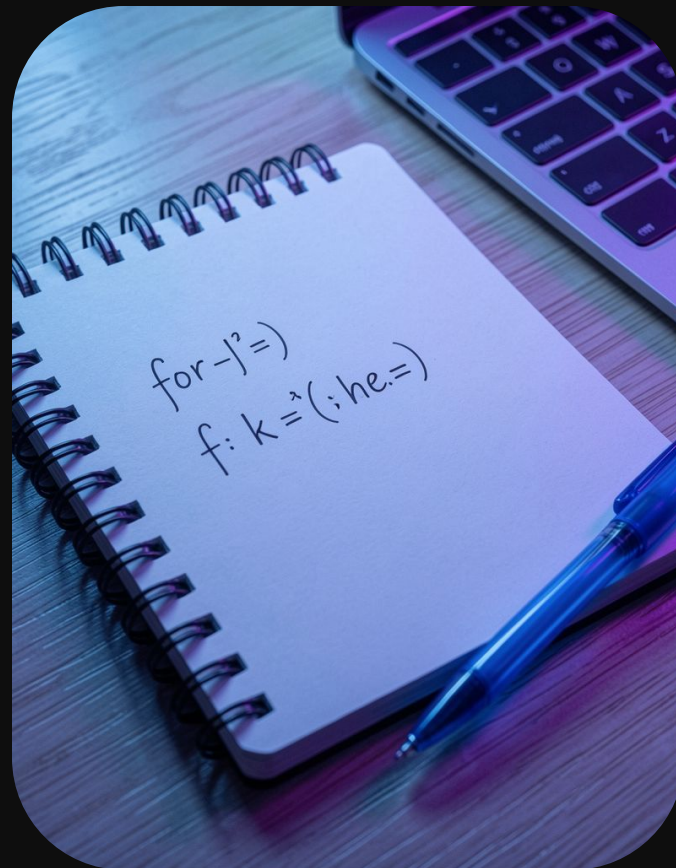
Introdução

Este módulo aborda laços *For*, *While* e iteração *Foreach*, além de métodos e parâmetros.

Objetivo: compreender sintaxe, controle de fluxo e boas práticas.

01

Laços For



Sintaxe básica

Use a estrutura *for* para iterações controladas por contador.

Exemplo: inicialização; condição; incremento.

Fácil de prever ideal para arrays indexados.



Exemplos práticos

Iterar arrays, somar valores, gerar sequências.

Mostrar código curto com contador e ações por iteração.

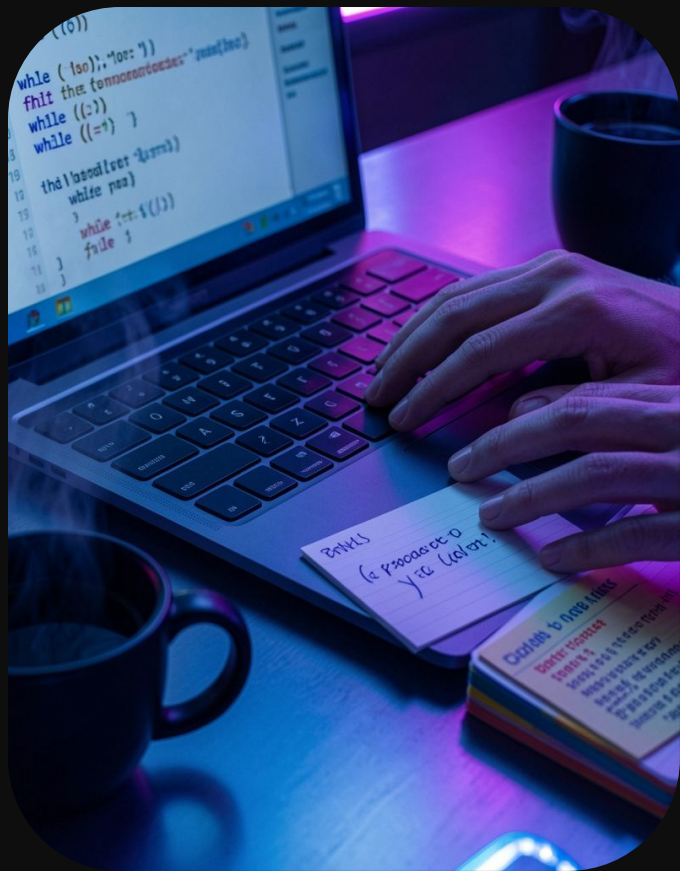
Aplicação direta em laços determinísticos.

Uso avançado

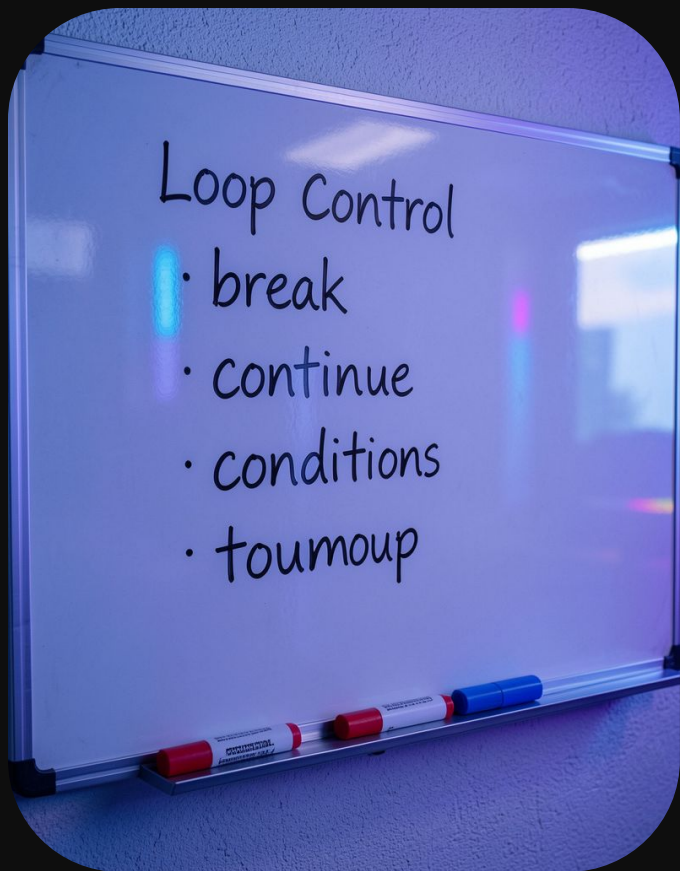
Aninhar laços, otimizar condições, usar break/continue.

Cuidado com complexidade $O(n^2)$ e legibilidade.

Prefira funções auxiliares para lógica complexa.



02 Laços While



Condições e controle

Evitar loops infinitos usando verificações e timeouts.

Empregar `break` para saída antecipada e `continue` para pular iterações.

Validar pré-condições antes do laço.

Evitar loop infinito

Verifique condições de término antes de entrar no laço e atualize variáveis de controle dentro do bloco.

Use *contadores*, limites máximos ou um *timeout* para segurança.

Empregue *break* para saída controlada e testes unitários para detectar loops indesejados.

03

Laços Foreach

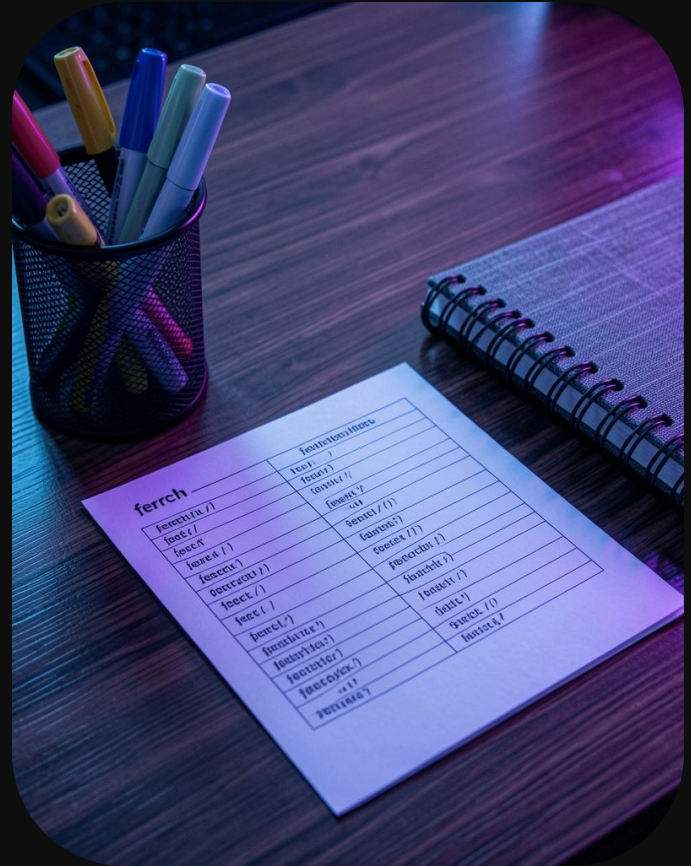
ços Foreach

Sintaxe em coleções

Oforeach percorre coleções sem expor índices.

Sintaxe simples: para cada elemento faça ações diretas.

Ideal para listas, arrays e dicionários.



Iteração segura

Evite modificar a coleção enquanto itera; copie ou use iteradores seguros.

Trate elementos nulos e exceções durante a iteração.

Prefira métodos imutáveis quando concorrência for possível.

Exemplos com arrays/listas

Iterar para somar valores, filtrar ou mapear resultados.

Exemplo curto: percorrer lista e acumular em variável.

Use *lambda* ou funções auxiliares para operações repetitivas.

04

Métodos e Parâmetros

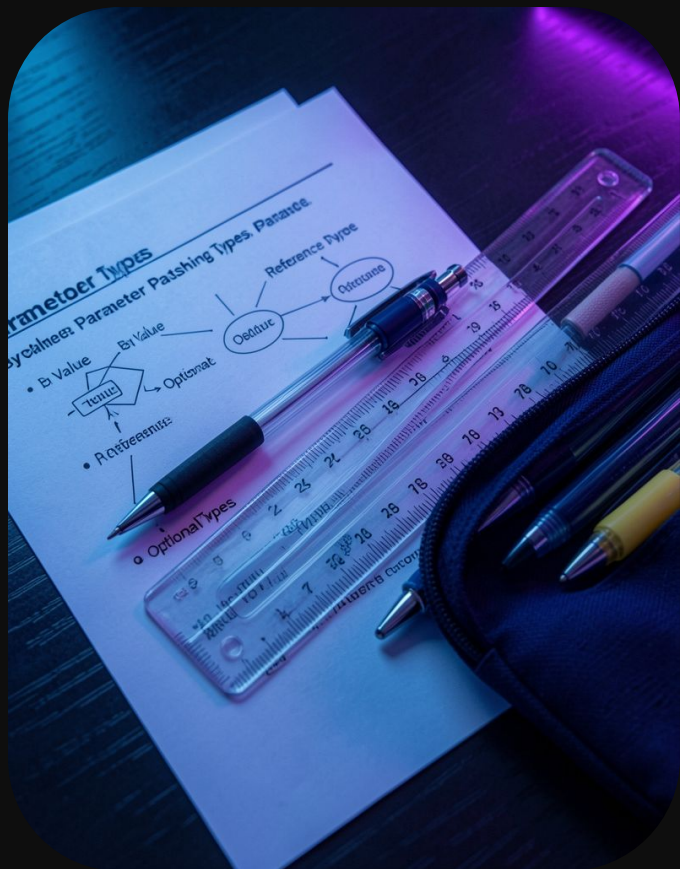


Definição e assinatura

Método: bloco nomeado que executa tarefa específica.

Assinatura inclui nome, parâmetros e tipo de retorno.

Mantenha responsabilidade única e nomes descritivos.



Tipos de parâmetros

Parâmetros por valor, referência e opcionais.

Use *params* ou padrões para flexibilidade.

Documente expectativas e validações de entrada.

Retorno e boas práticas

Retorne valores claros ou objetos de resultado para erros.

Evite múltiplos efeitos colaterais; prefira valores de retorno consistentes.

Implemente tratamento de exceções e contratos simples.

Conclusões

Resumo rápido: escolha `for` para contadores previsíveis, *while* para condições incertas e *foreach* para coleções.

Projete métodos claros com parâmetros validados e retornos consistentes.

Implemente proteções contra loops infinitos e mantenha código legível.

THANKS!

Do you have any questions?

+55 49 98892-8606

